TECNOLÓGICO
NACIONAL DE MÉXICO

Algebra relacional:

Gabriela Natalia De León Alvarez

I24050006

Ingeniería Informática

1. Marco Teórico

1.1 ¿Qué es el álgebra relacional?

El álgebra relacional es un lenguaje formal y procedimental para consultar bases de datos relacionales: se indica cómo obtener el resultado combinando operadores. Fue propuesta por Codd (1970, 1972) y constituye la base teórica del lenguaje SQL (Elmasri & Navathe, 2016; Silberschatz et al., 2010).

- **Cierre:** Cada operador recibe una o dos relaciones y devuelve otra relación, lo que permite anidar operaciones sucesivamente como π_nombre(σ_precio_venta>20(PRODUCTO)).
- **Semántica de conjuntos:** En el álgebra pura una relación es un conjunto estricto de tuplas (sin duplicados). SQL trabaja por defecto con multiconjuntos (admite duplicados) a menos que se especifique DISTINCT.
- **Operadores fundamentales:** Selección, proyección, unión, diferencia, producto cartesiano y renombrado. Los demás (intersección, reunión y división) se pueden derivar de ellos.

1.2 Clasificación de los operadores

Tipo de Operador	Operadores Comprendidos
Unarios (una sola relación)	Selección (σ), Proyección (π), Renombrado (ρ)
Binarios (dos relaciones)	Unión (∪), Intersección (∩), Diferencia (−), Producto cartesiano (×), Reunión (⋈), División (÷)
Extendidos	Proyección generalizada, Agregación ($\mathcal{G}$), Reuniones externas (⟕, ⟖, ⟗)

2. Mapa Conceptual

A continuación, se presenta la estructura conceptual jerárquica del álgebra relacional (disponible en formato editable draw.io, vectorial SVG y código Mermaid):

> **[ESTRUCTURA DEL MAPA CONCEPTUAL]** *ÁLGEBRA RELACIONAL (Lenguaje formal y procedimental con propiedad de cierre)*
> ├── *1. OPERADORES UNARIOS (Actúan sobre una sola relación)*
> │ ├── *σ Selección (σ_p(R): Filtra filas que cumplen predicado p)*
> │ ├── *π Proyección (π_A1..An(R): Selecciona columnas y elimina duplicados)*
> │ └── *ρ Renombrado (ρ_S(B1..Bn)(R): Cambia nombre de relación o atributos)*
> │
> ├── *2. OPERADORES BINARIOS (Actúan sobre dos relaciones)*
> │ ├── *TEORÍA DE CONJUNTOS (Requieren compatibilidad de unión)*
> │ │ ├── *∪ Unión (R ∪ S: Tuplas en R o en S)*
> │ │ ├── *∩ Intersección (R ∩ S: Tuplas en R y en S)*
> │ │ └── *− Diferencia (R − S: Tuplas en R pero no en S)*
> │ └── *COMBINACIÓN DE RELACIONES*
> │ ├── *× Producto Cartesiano (R × S: Todas las combinaciones posibles)*
> │ ├── *⋈ Reunión / Join (R ⋈_ϑ S: R × S filtrado por condición ϑ)*
> │ └── *÷ División (R ÷ S: Valores de R asociados a todas las tuplas de S)*
> │
> └── *3. ÁLGEBRA EXTENDIDA (Amplía el poder expresivo)*
> ├── *π Proyección Generalizada (Permite expresiones aritméticas)*
> ├── *𝒢 Agregación (_G1..Gn 𝒢 _F1..Fm(R): Agrupa y aplica sum, avg, min, max, count)*
> └── *REUNIONES EXTERNAS (Conservan tuplas sin pareja con valores NULL)*
> ├── *⟕ Reunión Externa Izquierda (Conserva tuplas de R)*
> ├── *⟖ Reunión Externa Derecha (Conserva tuplas de S)*
> └── *⟗ Reunión Externa Completa (Conserva tuplas de R y S)*

3. Relaciones de Ejemplo (Tienda de Abarrotes)

Para los ejemplos prácticos, se abrevian los códigos de barras como P1 a P4. Las relaciones utilizadas provienen del esquema del proyecto de la tienda de abarrotes:

PRODUCTO

codigo_barras	nombre	costo_compra	precio_venta	stock_actual	id_categoria
P1	Refresco 600 ml	10.00	15.00	20	1
P2	Arroz 1 kg	18.00	25.00	8	2
P3	Frijol 1 kg	22.00	30.00	12	2
P4	Aceite 1 L	35.00	45.00	3	2

CATEGORIA

id_categoria	nombre_cat
1	Bebidas
2	Abarrotes
3	Limpieza

DETALLE_VENTA

folio	codigo_barras	cantidad
1	P1	2
1	P2	1
2	P3	3
2	P1	1
2	P2	2
3	P2	1

ABASTECIMIENTO

id_proveedor	codigo_barras	costo_unitario
1	P1	9.00
1	P2	16.50
1	P4	33.00

4. Operadores Unarios

4.1 Selección (σ)

Notación formal: σ_p(R) = { t | t ∈ R ∧ p(t) }, donde p es un predicado lógico con comparaciones (=, ≠, <, ≤, >, ≥) combinadas mediante ∧, ∨, ¬.
Significado: Devuelve las filas de la relación R que satisfacen el predicado p. Conserva todos los atributos originales (grado no cambia, cardinalidad es ≤ |R|).
Ejemplo: Productos con precio de venta mayor a 20.00: **σ_{precio_venta > 20}(PRODUCTO)**

codigo_barras	nombre	costo_compra	precio_venta	stock_actual	id_categoria
P2	Arroz 1 kg	18.00	25.00	8	2
P3	Frijol 1 kg	22.00	30.00	12	2
P4	Aceite 1 L	35.00	45.00	3	2

> **Equivalencia SQL:** *SELECT * FROM producto WHERE precio_venta > 20;*

4.2 Proyección (π)

Notación formal: π_{A1, ..., An}(R) = { t[A1, ..., An] | t ∈ R }
Significado: Devuelve únicamente las columnas indicadas y elimina automáticamente las filas duplicadas (debido a la semántica de conjuntos).
Ejemplo 1: π_{nombre, precio_venta}(PRODUCTO)

nombre	precio_venta
Refresco 600 ml	15.00
Arroz 1 kg	25.00
Frijol 1 kg	30.00
Aceite 1 L	45.00

Eliminación de duplicados (Ejemplo 2): π_{id_categoria}(PRODUCTO) produce sólo 2 filas únicas a partir de los 4 productos:

id_categoria
1
2

Composición de operadores: Nombres de productos de la categoría 2: π_{nombre}(σ_{id_categoria=2}(PRODUCTO))

nombre
Arroz 1 kg
Frijol 1 kg
Aceite 1 L

> **Equivalencia SQL:** *SELECT DISTINCT nombre, precio_venta FROM producto;*

4.3 Renombrado (ρ)

Notación formal: ρ_{S(B1, ..., Bn)}(R) asigna el nombre de relación S y los nombres de atributos B1, ..., Bn a la relación R.
Significado: No altera la información de las tuplas; únicamente modifica la identidad o nombres de la estructura. Es esencial para autouniones o garantizar compatibilidad de unión.
Ejemplo: ρ_{ARTICULO(articulo, precio)}(π_{nombre, precio_venta}(σ_{id_categoria=1}(PRODUCTO)))

articulo	precio
Refresco 600 ml	15.00

> **Equivalencia SQL:** *SELECT nombre AS articulo, precio_venta AS precio FROM producto WHERE id_categoria = 1;*

5. Operadores Binarios

5.1 Compatibilidad de Unión

Para los operadores ∪, ∩ y −, las relaciones R y S deben ser **compatibles de unión**: deben poseer el mismo número de atributos (mismo grado) y los dominios de los atributos correspondientes deben ser idénticos. El resultado hereda los nombres de atributo de la primera relación.

Relaciones de prueba utilizadas:
• V = π_{codigo_barras}(DETALLE_VENTA) = { P1, P2, P3 } (Productos vendidos)
• A = π_{codigo_barras}(ABASTECIMIENTO) = { P1, P2, P4 } (Productos surtidos por proveedor)

5.2 Unión (∪)

Notación formal: R ∪ S = { t | t ∈ R ∨ t ∈ S }
Significado: Contiene todas las tuplas que están en R, en S o en ambas, eliminando duplicados. |R ∪ S| ≤ |R| + |S|.
Ejemplo (V ∪ A): Productos que se vendieron o que se surten.

codigo_barras
P1
P2
P3
P4

> **Equivalencia SQL:** *SELECT codigo_barras FROM detalle_venta*
> *UNION*
> *SELECT codigo_barras FROM abastecimiento;*

5.3 Intersección (∩)

Notación formal: R ∩ S = { t | t ∈ R ∧ t ∈ S }. Operador derivado: R ∩ S = R − (R − S).
Significado: Tuplas presentes en ambas relaciones simultáneamente.
Ejemplo (V ∩ A): Productos que se han vendido y que el proveedor también surte.

codigo_barras
P1
P2

> **Equivalencia SQL:** *SELECT codigo_barras FROM detalle_venta*
> *INTERSECT*
> *SELECT codigo_barras FROM abastecimiento;*

5.4 Diferencia (–)

Notación formal: R − S = { t | t ∈ R ∧ t ∉ S }. No es conmutativo (R − S ≠ S − R).
Significado: Tuplas presentes en R pero ausentes en S.
Ejemplo 1 (V – A): Productos vendidos que el proveedor NO surte → { P3 }
Ejemplo 2 (A – V): Productos surtidos pero que NO se han vendido → { P4 }

codigo_barras (V – A)
P3

codigo_barras (A – V)
P4

> **Equivalencia SQL:** *SELECT codigo_barras FROM detalle_venta EXCEPT SELECT codigo_barras FROM abastecimiento;*

5.5 Producto Cartesiano (×)

Notación formal: R × S = { t q | t ∈ R ∧ q ∈ S }
Significado: Combina cada tupla de R con cada tupla de S. Grado = Grado(R) + Grado(S), Cardinalidad = |R| × |S|.
Ejemplo: π_{nombre_cat}(CATEGORIA) × TURNO, con TURNO = {MATUTINO, VESPERTINO} (3 × 2 = 6 filas).

nombre_cat	turno
Abarrotes	MATUTINO
Abarrotes	VESPERTINO
Bebidas	MATUTINO
Bebidas	VESPERTINO
Limpieza	MATUTINO
Limpieza	VESPERTINO

> **Equivalencia SQL:** *SELECT c.nombre_cat, t.turno FROM categoria c CROSS JOIN turno t;*

5.6 Reunión / Join (⋈)

• **Reunión Theta:** R ⋈_θ S = σ_θ(R × S). Producto cartesiano filtrado por una condición θ.
• **Equirreunión:** Caso de reunión Theta donde la condición θ utiliza únicamente operadores de igualdad (=).
• **Reunión Natural:** R ⋈ S iguala automáticamente los atributos homónimos en R y S y suprime columnas duplicadas.

Ejemplo (Reunión Natural PRODUCTO ⋈ CATEGORIA):

codigo_barras	nombre	id_categoria	nombre_cat
P1	Refresco 600 ml	1	Bebidas
P2	Arroz 1 kg	2	Abarrotes
P3	Frijol 1 kg	2	Abarrotes
P4	Aceite 1 L	2	Abarrotes

Ejemplo (Reunión Theta): PRODUCTO ⋈_{PRODUCTO.codigo_barras = ABASTECIMIENTO.codigo_barras ∧ precio_venta > 1.5 * costo_unitario} ABASTECIMIENTO

nombre	precio_venta	costo_unitario
Arroz 1 kg	25.00	16.50
Refresco 600 ml	15.00	9.00

> **Equivalencia SQL:** *SELECT p.nombre, p.precio_venta, a.costo_unitario*
> *FROM producto p*
> *JOIN abastecimiento a ON p.codigo_barras = a.codigo_barras*
> *WHERE p.precio_venta > 1.5 * a.costo_unitario;*

5.7 División (÷)

Notación formal: Dado R(X, Y) y S(Y), R ÷ S = { t[X] | t ∈ R ∧ ∀s ∈ S ((t[X], s) ∈ R) }
Fórmula en operadores fundamentales: R ÷ S = π_X(R) − π_X((π_X(R) × S) − R)
Significado: Devuelve los valores de X que están asociados con TODOS los valores del conjunto S

(consultas tipo 'para todo').
Ejemplo: Clientes que compraron TODOS los productos en PROMOCION = { P1, P2 }.

id_cliente	codigo_barras (COMPRA_CLIENTE)
1	P1
1	P2
2	P1
2	P2
2	P3
3	P1

Resultado COMPRA_CLIENTE ÷ PROMOCION:

id_cliente
1
2

(El cliente 3 no aparece porque solo compró P1 y le faltó P2).

> **Equivalencia SQL (Doble NOT EXISTS):** *SELECT DISTINCT c.id_cliente*
> *FROM compra_cliente c*
> *WHERE NOT EXISTS (*
> *SELECT 1 FROM promocion p*
> *WHERE NOT EXISTS (*
> *SELECT 1 FROM compra_cliente c2*
> *WHERE c2.id_cliente = c.id_cliente AND c2.codigo_barras = p.codigo_barras*
> *)*
> *);*

6. Álgebra Relacional Extendida

Los operadores básicos son limitados al no poder realizar cálculos numéricos, agregaciones sobre grupos o retener información incompleta. El álgebra extendida soluciona estas deficiencias (Silberschatz et al., 2010).

6.1 Proyección Generalizada

Notación formal: π_{F1, F2, ..., Fn}(R), donde cada Fi es una expresión aritmética involucrando atributos y constantes.
Significado: Extiende la proyección estándar permitiendo calcular y generar nuevas columnas derivadas.
Ejemplo (Ganancia bruta por producto): π_{nombre, precio_venta − costo_compra → margen}(PRODUCTO)

nombre	margen
Refresco 600 ml	5.00
Arroz 1 kg	7.00
Frijol 1 kg	8.00
Aceite 1 L	10.00

> **Equivalencia SQL:** *SELECT nombre, (precio_venta - costo_compra) AS margen FROM producto;*

6.2 Funciones de Agregación ($\mathcal{G}$)

Notación formal: _{G1, ..., Gn} $\mathcal{G}$ _{F1(A1), ..., Fm(Am)}(R), donde G1..Gn son los atributos de agrupación y Fi las funciones agregadas (sum, avg, min, max, count).
Significado: Agrupa las tuplas compartiendo valores en G y calcula métricas resumen por grupo.
Ejemplo 1 (Con agrupación): _{id_categoria} $\mathcal{G}$ _{count(codigo_barras), avg(precio_venta)}(PRODUCTO)

id_categoria	num_productos	precio_promedio
1	1	15.00
2	3	33.33

Ejemplo 2 (Sin agrupación / Global): $\mathcal{G}$ _{sum(stock_actual)}(PRODUCTO)

total_stock
43

> **Equivalencia SQL:** *SELECT id_categoria, COUNT(codigo_barras) AS num_productos, AVG(precio_venta) AS precio_promedio*
> *FROM producto*
> *GROUP BY id_categoria;*

6.3 Reuniones Externas (Outer Joins)

Conservan las tuplas que no satisfacen la condición de acoplamiento, rellenando los atributos faltantes con NULL.

Operación	Notación	Comportamiento / Conservación
Externa Izquierda (Left Outer Join)	R ⟕_θ S	Conserva TODAS las tuplas de R; rellena con NULL si no hay pareja en S.
Externa Derecha (Right Outer Join)	R ⟖_θ S	Conserva TODAS las tuplas de S; rellena con NULL si no hay pareja en R.
Externa Completa (Full Outer Join)	R ⟗_θ S	Conserva TODAS las tuplas de R y de S, rellenando con NULL según corresponda.

Ejemplo Izquierda (CATEGORIA ⟕ PRODUCTO): La categoría 'Limpieza' se conserva aunque no tenga productos asociados:

id_categoria	nombre_cat	codigo_barras	nombre
1	Bebidas	P1	Refresco 600 ml
2	Abarrotes	P2	Arroz 1 kg
2	Abarrotes	P3	Frijol 1 kg

2	Abarrotes	P4	Aceite 1 L
3	Limpieza	NULL	NULL

> **Equivalencia SQL (Left Join):** *SELECT c.id_categoria, c.nombre_cat, p.codigo_barras, p.nombre*
> *FROM categoria c*
> *LEFT JOIN producto p ON p.id_categoria = c.id_categoria;*

Ejemplo Completa (VENTAS ⟕ ABASTECIMIENTO): Donde VENTAS = _{codigo_barras} $\mathcal{G}$ _{sum(cantidad)}(DETALLE_VENTA). P3 sólo fue vendido; P4 sólo fue surtido:

codigo_barras	cantidad_vendida	costo_unitario
P1	3	9.00
P2	4	16.50
P3	3	NULL
P4	NULL	33.00

> **Equivalencia SQL (Full Join):** *SELECT COALESCE(v.codigo_barras, a.codigo_barras) AS codigo_barras, v.cantidad_vendida, a.costo_unitario*
> *FROM (...) v*
> *FULL OUTER JOIN abastecimiento a ON a.codigo_barras = v.codigo_barras;*

7. Cuadro Resumen de Operadores

Operador	Símbolo	Aridad	Notación	Resultado / Propósito	Equivalente SQL
Selección	σ	Unario	σ_p(R)	Filas que cumplen el predicado p	WHERE
Proyección	π	Unario	π_{A1..An}(R)	Columnas elegidas sin duplicados	SELECT DISTINCT
Renombrado	ρ	Unario	ρ_{S(B1..Bn)}(R)	Misma relación con nuevos nombres	AS
Unión	∪	Binario	R ∪ S	Tuplas en R o en S (compatibles)	UNION
Intersección	∩	Binario	R ∩ S	Tuplas presentes en R y en S	INTERSECT
Diferencia	−	Binario	R − S	Tuplas en R que no están en S	EXCEPT
Producto Cartesiano	×	Binario	R × S	Todas las combinaciones posibles	CROSS JOIN
Reunión (Join)	⋈	Binario	R ⋈_θ S	σ_θ(R × S) por condición	JOIN ... ON
División	÷	Binario	R ÷ S	Valores de R asociados a todo S	Doble NOT EXISTS
Proyección Generalizada	π	Unario	π_{F1..Fn}(R)	Columnas calculadas expresionales	SELECT expresión
Agregación	$\mathcal{G}$	Unario	_{G}$\mathcal{G}$_{F(A)}(R)	Una fila por grupo con métricas	GROUP BY + aggregate

Reunión Externa	⟕ ⟖ ⟗	Binario	R ⟗_θ S	Conserva tuplas sin pareja (NULL)	LEFT/RIGHT/FULL JOIN

8. Operadores Derivados

Algunos operadores algebraicos pueden ser expresados exactamente mediante combinaciones de los operadores fundamentales, por lo que no añaden poder expresivo estricto al álgebra pero simplifican la sintaxis:

Operador Derivado	Expresión Formal con Operadores Fundamentales
Intersección (∩)	$R \cap S = R - (R - S)$
Reunión Theta (⋈_θ)	$R \bowtie_\theta S = \sigma_\theta(R \times S)$
División (÷)	$R \div S = \pi_X(R) - \pi_X((\pi_X(R) \times S) - R)$

Referencias Bibliográficas

Codd, E. F. (1970). A relational model of data for large shared data banks. Communications of the ACM, 13(6), 377-387. https://doi.org/10.1145/362384.362685

Codd, E. F. (1972). Relational completeness of data base sublanguages. En R. Rustin (Ed.), Data base systems (pp. 65-98). Prentice-Hall.

Date, C. J. (2004). An introduction to database systems (8.ª ed.). Pearson/Addison-Wesley.

Elmasri, R., & Navathe, S. B. (2016). Fundamentals of database systems (7.ª ed.). Pearson.

Silberschatz, A., Korth, H. F., & Sudarshan, S. (2010). Database system concepts (6.ª ed.). McGraw-Hill.